New chat
My stuff
Gems
Chats
Given A String check whether an reve...
વોલ્યુમ 1ની સંપૂર્ણતા પર જાઓ
Image To Video Generation
Sort Array By Binary Bit Count
Random Index Matrix Sums
Sort Array By Bit Count

String Methods

Given A String check whether an reversal is the same or not.
Return True If_ given otherwise return False.

Example 1

Input:- word = "madam"
Output:- true

Use of Javascript for loop. Understand step by step
Gujarati language

Show thinking

વોલ્યુમ, અને "માનવીય (Javascript) જેવું" સ્ટ્રિંગ દેખીતી રીતે કોઈપણ ઉલટાવેલ નથી કારણ કે તે પછી (Palindrome Check) તે સમાનતા ટેસ્ટ છે. જો ટ્રાન્સફોર્મ કરેલ અને મૂળ સમાન હોય.

તો તે સમજવું છે (Step-by-Step Explanation in Gujarati)
આ વોલ્યુમ સ્ટ્રિંગ કામ કરવા માટેની ઉલટાવેલ સ્ટ્રિંગ સાથે સરખામણી કરવામાં આવે છે.

1. મૂળભૂત (Setup): અમારી ઉલટાવેલ સ્ટ્રિંગ (reversed) અને મૂળ સ્ટ્રિંગ (original) સ્ટ્રિંગ બનાવો.
2. સરખામણી (Comparison): અમારી ઉલટાવેલ સ્ટ્રિંગની મૂળ સ્ટ્રિંગ સાથે સરખામણી કરો (reversed === original).
3. પરિણામ (Result): જો સ્ટ્રિંગ મેચ થાય છે, તો અમારી તપાસ પરત કરવી જોઈએ true.
4. જો તફાવત (Result) મેચ ન કરતા હોય, તો અમારી તપાસ પરત કરવી જોઈએ false.

Example: madam (મેડમ) પર ઉલટાવેલ સ્ટ્રિંગ madam (મેડમ) બરાબર છે, તો true પરત કરો.

Javascript Code

```
function checkPalindromeLoop(str) {
    let len = str.length;

    // ઉલટાવેલ સ્ટ્રિંગ માટે ખાલી સ્ટ્રિંગ
    for (let i = 0; i < len; i++) {
        // દરેક અક્ષરને મૂળ સ્ટ્રિંગના છેલ્લાથી શરૂ કરીને ઉમેરો
        let reversedChar = str[len - 1 - i];
        if (reversedChar !== str[i]) {
            return false;
        }
    }

    // જો લૂપ પૂરો થઈ ગયો અને બધાં અક્ષરો સમાન હોય, તો true રિટર્ન કરો
    return true;
}

// ઉદાહરણ ઉપયોગ
let inputWord = "madam";
console.log(checkPalindromeLoop(inputWord));
```

ઉદાહરણ માટે કામનું છે (Pro Run with "madam")
ચાલો તે કરવા "madam" ટેસ્ટ કરીએ તે છે.

- Step 1 (i = 0): str[0] = 'm'
  - અક્ષરનો અર્થ: str[4] = 'm'
  - તે સમાન છે ('m' == 'm'), તેથી લૂપ ચાલતો રહેશે.
- Step 2 (i = 1):
  - અક્ષરનો અર્થ: str[3] = 'a'
  - અક્ષરનો અર્થ: str[1] = 'a'
  - તે સમાન છે ('a' == 'a'), તેથી લૂપ ચાલતો રહેશે.
- Step 3 (i = 2):
  - અક્ષરનો અર્થ: str[2] = 'd'
  - અક્ષરનો અર્થ: str[2] = 'd'
  - તે સમાન છે ('d' == 'd'), તેથી લૂપ ચાલતો રહેશે.
  - લૂપ પૂરો થઈ ગયો છે, કારણ કે તેણે પોતાને અડધે રસ્તે પાર કર્યું છે.
- Final Output: true: true રિટર્ન કરો.

તમે જોઈ શકો છો કે સ્ટ્રિંગમાં કોઈ એક જોડી ઉલટાવેલ કરવાની જરૂર નથી તેથી તમામ ઉલટાવેલ જોડી મેચ થઈ છે!

Find max. num. of String

A sentence is a list of words that are separated by a single space with no leading or trailing spaces. You are given an array of strings sentences, where each sentences[i] represents a single sentence. Return the maximum number of words that appear in a single sentence.

Example 1:
Input: sentences = ["alice and bob love leetcode", "i think so too", "this is great thanks very much"]
Output: 6

use vs_anonymous for loop. understanding step by step.
Gujarati language

Show thinking

વોલ્યુમ, અને "માનવીય (Javascript) જેવું" સ્ટ્રિંગ દેખીતીને કોઈ વોર્ડ નથી. તે ખાલી અવકાશ સાથેનો એક સ્ટ્રિંગ છે.

(Maximum Number of Words સંબંધિત) તે છે.

તો તે સમજવું છે (Step-by-Step Explanation in Gujarati)
આ વોલ્યુમ સ્ટ્રિંગ કામ કરવા માટેનો ઉકેલ તમામ વાક્યો દ્વારા તેમાં રહેલા શબ્દોની સંખ્યાને માપવાનો છે અને પછી તેમાંથી મહત્તમ શોધવાનો છે.

1. મૂળભૂત (Setup): વોર્ડ કાઉન્ટ (maxWords) નામનો એક વેરિએબલ બનાવો અને તેને શૂન્ય (0) તરીકે સેટ કરો.
2. લૂપ (Looping): સ્ટ્રિંગમાંના દરેક વાક્ય (sentence) માટે એક વેરિએબલ બનાવવા માટે for-loop નો ઉપયોગ કરો.
   - વર્ડ કાઉન્ટ નક્કી કરો (Counting Words): દરેક વાક્યને સ્પેસ (' ') દ્વારા વિભાજિત કરો અને પરિણામી એરેની લંબાઈ ગણો (sentence.split(' ').length).
   - મહત્તમને અપડેટ કરો (Update Max): જો વર્તમાન વાક્યમાં શબ્દોની સંખ્યા અગાઉના મહત્તમ કરતાં વધુ હોય, તો maxWords ને અપડેટ કરો.
3. પરિણામ (Result): લૂપ પૂરો થયા પછી, maxWords કિંમત પરત કરો.

Javascript Code

```
function maxWordsCount(sentences) {
    // મહત્તમ શબ્દોની ગણતરી માટે શરૂઆતી વેરિએબલ
    let maxWords = 0;

    // વાક્યો દ્વારા લૂપ કરો
    for (let i = 0; i < sentences.length; i++) {

        let currentSentence = sentences[i];

        // વર્તમાન વાક્યમાં શબ્દોની સંખ્યાની ગણતરી કરો
        // સ્પેસ પર વિભાજીત કરો અને પરિણામી એરેની લંબાઈ મેળવો
        let currentWordCount = currentSentence.split(' ').length;

        // જો આ વાક્ય અગાઉના મહત્તમ કરતાં વધુ શબ્દો ધરાવે છે, તો મહત્તમને અપડેટ કરો
        if (currentWordCount > maxWords) {
            maxWords = currentWordCount;
        }
    }

    // જે. એરેમાં શબ્દોની સંખ્યા રિટર્ન કરો
    return maxWords;
}

// ઉદાહરણ ઉપયોગ
let sentences = ["alice and bob love leetcode", "i think so too", "this is great thanks very much"];
let result = maxWordsCount(sentences);

console.log(result); // Output: 6
```

ઉદાહરણ કામ કરવાનું છે (Dry Run)
Input: ["alice and bob love leetcode", "i think so too", "this is great thanks very much"]

- Step 1 (i=0): "alice and bob love leetcode"
  - શબ્દો: ["alice", "and", "bob", "love", "leetcode"]
  - વર્તમાનની ગણતરી: 5
  - maxWords: અપડેટ થયેલ છે. 5
- Step 2 (i=1): "i think so too"
  - શબ્દો: ["i", "think", "so", "too"]
  - વર્તમાનની ગણતરી: 4
  - maxWords: અપડેટ થયેલ નથી. maxWords હજુ પણ 5 છે.
- Step 3 (i=2): "this is great thanks very much"
  - શબ્દો: ["this", "is", "great", "thanks", "very", "much"]
  - વર્તમાનની ગણતરી: 6
  - maxWords: અપડેટ થયેલ છે. હવે, maxWords 6 છે.
- Final Output: 6 પૂર્ણ થઈ ગયું છે. હવે પરિણામ 6 પરત કરો.

શું તમે ઈચ્છો છો કે હું આ કોડને reduce એક્સપ્રેશન એક્સપ્રેશન, Functional પ્રોગ્રામિંગ દૃષ્ટિકોણ સાથે પણ સમજાવું?

You have given a string. You have to add characters at start of string to make it a palindrome.
return the minimum number of characters required to add to make it a palindrome.

Example 1
Input:
str = "abcd"
Output: 3

Show thinking

આ સોલ્યુશન સ્ટ્રિંગ દીઠ જરૂરી ઓછામાં ઓછાં અક્ષરો ઉમેરવાનું ન્યૂનતમ કાર્યક્ષમ અભિગમ છે. તેનો ઉપયોગ Palindrome Longest Palindromic Prefix સંબંધિત સમસ્યાઓ માટે પણ થઈ શકે છે.

તો તે સમજવું છે (Logic in Gujarati)
આ અલ્ગોરિધમ સ્ટ્રિંગ કામ કરવાનું છે.

1. અમારી ઉલટાવેલ સ્ટ્રિંગ બનાવો (Reverse): મૂળ સ્ટ્રિંગનું એક ઉલટાવેલ સ્ટ્રિંગ બનાવો.
2. અમારી વર્તમાન સ્ટ્રિંગ શોધો: અમારી વર્તમાન સ્ટ્રિંગ શોધો. તે શરૂઆતમાં મૂળ સ્ટ્રિંગ જેવી જ હશે.
3. મેચ અને ઘટાડો: અમારી મેચ થતા અક્ષરો શોધવા માટે લૂપ ચલાવો.
   - મુખ્ય: જ્યારે તમે મેચ શોધો, ત્યારે પ્રથમ Palindrome બંધ કરો.
4. પરિણામ: અક્ષરોની સંખ્યા પરત કરો.

Javascript Code

```
function minCharsToMakePalindrome(str) {
    let n = str.length;

    // ઉલટાવેલ સ્ટ્રિંગમાંથી પ્રારંભિક સ્ટ્રિંગ બનાવો
    let revStr = str.split('').reverse().join('');

    // વર્તમાન સ્ટ્રિંગ શોધો જે શરૂઆતમાં મૂળ સ્ટ્રિંગ જેવી જ હશે
    let currentStr = str;

    // મેચ થતા અક્ષરોની સંખ્યાની ગણતરી કરો
    let matchCount = 0;

    // મેચ થતા અક્ષરો શોધવા માટે લૂપ ચલાવો
    for (let i = 0; i < n; i++) {
        if (currentStr[i] === revStr[i]) {
            matchCount = i + 1;
        }
        else {
            break;
        }
    }

    return n - matchCount;
}

// ઉદાહરણ ઉપયોગ
let str = "abcd";
console.log(minCharsToMakePalindrome(str));
```

ચાલો ચલાવો કરો: કામનું છે (Dry Run with "abcd")
ઇનપુટ: "abcd" (n = 4)

- Step 1 (i = 0):
  - શું "abcd" છે? શું તે Palindrome છે? → ના.
  - મુખ્ય અક્ષરો બાકી (ઉલટાવેલ કરવાના અક્ષરો).
- Step 2 (i = 1):
  - શું "aba" છે? શું તે Palindrome છે? → ના.
- Step 3 (i = 2):
  - શું "abc" છે? શું તે Palindrome છે? → ના.
- Step 4 (i = 3):
  - શું "a" છે? શું તે Palindrome છે? → હા.
  - Palindrome મળ્યો! તપાસ બંધ કરો.
  - પરિણામ = 4 - 1 = 3 (ઉમેરવાનું અક્ષરોની સંખ્યા).

શું તમે ઈચ્છો છો કે હું આ કોડને reduce અથવા Functional પ્રોગ્રામિંગ દૃષ્ટિકોણ સાથે પણ સમજાવું છું?

var minimum_count = function() {

var code in this

Show thinking

બરાબર, હવે તમે મિનિમમ કાઉન્ટ ફંક્શન var minimum_count = function() માં અક્ષરોનો ઉમેરો કરો.

તો તે સમજવું છે (Gujarati Explanation)
આ વોલ્યુમ સ્ટ્રિંગ કામ કરવાનું છે.

1. મૂળભૂત (Setup): અમારી ઉલટાવેલ સ્ટ્રિંગમાંથી શરૂઆતમાં એક ખાલી સ્ટ્રિંગ બનાવો અને તેમાં અક્ષરોનો ઉમેરો કરો પછીથી આગળ Palindrome શોધો.
2. લૂપ (Looping): અમારી ઉલટાવેલ સ્ટ્રિંગમાંના દરેક અક્ષર માટે લૂપ ચલાવો.
3. તપાસ (Check): અમારી હાલની સ્ટ્રિંગ મેચ છે કે નહીં તે જુઓ કે શું તે અમારા ઉલટાવેલ સ્ટ્રિંગ સાથે Palindrome છે?
4. પરિણામી (Calculation): જેમાં અમારો Palindrome મળ્યો નથી, અમારી રૂપે ઉલટાવેલ સ્ટ્રિંગમાંથી અક્ષરોની સંખ્યાની ગણતરી કરો અને તેને કુલ લંબાઈમાંથી બાદબાકી કરીને.

Javascript Code

```
var minimum_count = function() {
    var n = s.length;

    // આ એક નવું ખાલી સ્ટ્રિંગ છે જે પછી 1 થી n સુધી પ્રત્યેક અક્ષર Palindrome છે કે નહીં તે તપાસશે.
    var result = "";

    for (var i = 1; i <= n; i++) {
        var left = 0;
        var right = i - 1;
        var isPalindrome = true;

        while (left < right) {
            if (s[left] !== s[right]) {
                isPalindrome = false;
                break;
            }
            left++;
            right--;
        }

        // જો આપણે સ્ટ્રિંગનો અડધો ભાગ તપાસ્યો છે, તો આપણે Palindrome શોધી શકીએ છીએ.
        if (isPalindrome) {
            // આ સ્ટ્રિંગ માટે, અમારી પ્રારંભિક સ્ટ્રિંગ મેળવો (i + 1) થી n સુધી.
            var remaining = s.substring(i, n);
            return remaining.length;
        }
    }

    return 0; // જો કોઈ પેલિન્ડ્રોમ ન મળે તો.
};

// ઉદાહરણ ઉપયોગ
var s = "abcd";
console.log("Characters to add: " + minimum_count()());
// Output: 3
```

કોડ કામ કરતા જુઓ: ડ્રાય રન (Dry Run): "abcd"

- i = 1: શું "abcd" પેલિન્ડ્રોમ છે? → ના.
- i = 2: શું "abc" પેલિન્ડ્રોમ છે? Palindrome શોધો → ના.
- i = 3: શું "abcd" પેલિન્ડ્રોમ છે? Palindrome શોધો → ના.
- i = 4: શું "a" પેલિન્ડ્રોમ છે? Palindrome શોધો → હા. કોડ અહીં રોકાય છે. Palindrome રોકો.

અંતિમ ગણતરી:

- કુલ લંબાઈ: 4
- Palindrome તરીકે મળેલી લંબાઈ: 1
- જરૂરી અક્ષરો: 4 - 1 = 3

તમારી minimum_count() ફંક્શન અહીં છે. તમારી સ્ટ્રિંગ એક જ "a" છે તેથી તે "abcd" → "dabcd" ઉમેરી શકાય છે.

સ્ટ્રિંગ ઉમેરવામાં "dabcd"

- "dabcd" Palindrome શોધો → ના.
- "abcd" Palindrome શોધો → ના.
- Palindrome તપાસો: 1 - 1 = 0.
- જરૂરી અક્ષરો: 4 - 0 = 4.

Given A String Extract all numbers from it and store it inside an array. Return the Array Once extraction is completed.

Note that if the string is "abc2dfe4" then the number is 24 and not 2,4 as individual numbers

Example 1
Input:
str = "abc2dfe4dd"
Output: [24, 4]

Show thinking

બરાબર, અહીં "માનવીય જેવું" સ્ટ્રિંગ દેખીતીને કોઈ નિયમિત અભિવ્યક્તિ અથવા digit (Continuous Numbers) સ્ટ્રિંગમાંથી સતત અંકોનો ઉકેલ જરૂરી છે.

તો તે સમજવું છે (Step-by-Step Explanation in Gujarati)
આ વોલ્યુમ્સ્ટ્રિંગ કામ કરવાનું છે તે સ્ટ્રિંગમાંથી અંકોના સતત ક્રમને કાઢવાનું છે અને પછી તેમાંથી પૂર્ણાંકોની એક સૂચિ બનાવવાનું છે.

1. પ્રારંભિક (Initialize):
   - resultArr: અક્ષરોના અંકોના સંગ્રહ માટે ખાલી એરેનું સર્જન કરો.
   - tempNum: તમામ અક્ષરોના અંકો સંગ્રહ કરવાનું શૂન્ય (0) તરીકે શરૂઆત કરો.
2. લૂપ દ્વારા (Loop Through): સ્ટ્રિંગના દરેક અક્ષર માટે લૂપ ચલાવો.
3. તપાસો (Check):
   - જો અક્ષર ડિજીટ હોય તો: tempNum ને જોડીને (10 * n + digit) અપડેટ કરો.
   - જો અક્ષર ડિજીટ નહીં હોય તો: જો tempNum શૂન્ય કરતાં વધુ હોય, તો તેને resultArr માં ઉમેરો અને tempNum ને શૂન્ય પર રીસેટ કરો.
4. છેલ્લું પરિણામ ઉમેરો: લૂપ પૂરો થયા પછી, જો tempNum શૂન્ય કરતાં વધુ હોય તો તેને ઉમેરવા માટે તપાસ કરો, કારણ કે તે સ્ટ્રિંગ અંકોથી શરૂ થઈ શકે છે.
5. પરત કરો: અંતિમ પરિણામ પરત કરો.

Javascript Code

```
function extractNumbers(str = "") {
    let resultArr = [];     // અંકો માટે ખાલી એરે બનાવો
    let tempNum = "";       // અંકો માટે ટેમ્પ્લેટ સ્ટ્રિંગ

    for (let i = 0; i < str.length; i++) {
        let char = str[i];

        // જો વર્તમાન અક્ષર અંક છે, તો તેને ટેમ્પ્લેટ સ્ટ્રિંગમાં ઉમેરો.
        if (char >= '0' && char <= '9') {
            tempNum = tempNum + char;
        }
        // જો અક્ષર અંક નથી, તો ટેમ્પ્લેટ સ્ટ્રિંગ મેળવો.
        else {
            // જો ટેમ્પ્લેટ સ્ટ્રિંગ ખાલી નથી, તો તેને પરિણામમાં ઉમેરો.
            if (tempNum > 0) {
            resultArr.push(parseInt(tempNum));
            // ટેમ્પ્લેટને રીસેટ કરો.
            tempNum = "";
            }
        }
    }

    // છેલ્લું પરિણામ ઉમેરો, જો કોઈ હોય તો.
    if (tempNum > 0) {
        resultArr.push(parseInt(tempNum));
    }

    return resultArr;
}

// ઉદાહરણ ઉપયોગ
let input = "abc2dfe4";
console.log(extractNumbers(input));
// Output: [24, 4]
```

ઉદાહરણ કામ કરવાનું છે (Dry Run): "abc2dfe4dd"

- a, b, c: તે અક્ષરો છે. tempNum ખાલી છે અને કંઈ ઉમેરવામાં આવતું નથી.
- '2': અંક છે → tempNum = "2"
- 'd': અંક છે → tempNum = "22"
- 'f': અંક છે → tempNum = "222"
- 'e': અંક છે → tempNum = "2224"
- 'd': અંક નથી. તે પરિણામ ઉમેરવાનો સમય છે.
  - tempNum ને "2224" તરીકે પાર્સ કરો.
  - resultArr માં ઉમેરો. હવે resultArr [2224].
  - tempNum ને ખાલી તરીકે રીસેટ કરો.
- 'd': અંક નથી. tempNum ખાલી છે. કંઈ ઉમેરવું નથી.
- 'e': અંક નથી. tempNum ખાલી છે. કંઈ ઉમેરવું નથી.
- 'd': અંક નથી. tempNum ખાલી છે. કંઈ ઉમેરવું નથી.
- અંતિમ પરિણામ: [2224]

- `tempNum` ને ખાલી કરો ("").

6. **'4':** અંક છે -> `tempNum` = "4"
7. **'4':** અંક છે -> `tempNum` = "44"
8. **'3':** બિન અંક છે.
   - અત્યારે `tempNum` માં "44" છે.
   - તેથી 44 ને `result` એરેમાં નાખો. `[334, 44]`.
   - `tempNum` ને ખાલી કરો.

Final Output: `[334, 44]`

શું તમને `Number(tempNum)` કંઈક-નવી ઉમેરીને સમજાયો? તે સિદ્ધિને "334" ને નંબર `334` માં ફેરવે છે.

In A exam with negative marking the passing criteria is you have to score zero or positive marks .
we have given an array which denotes the score of the students in that exam .
you have to move all the failed students to last of the array .

Solve this Question in O(N) time complexity. Using two pointers. Watch Hints if not being able to figure out.

Example 1:

```
Input :
N = 8
arr[] = {1, -1, 3, 2, -7, -5, 11, 6 }
Output : 1 3 2 11 6 -1 -7 -5
```

Show thinking

લોજિક, આ પ્રોબ્લેમ "Move Zeroes to End" જેવો જ છે. અહીં આપણે પાસ થયેલા વિદ્યાર્થીઓ (Positive Numbers) ને આગળ લાવવાના છે અને નાપાસ થયેલા વિદ્યાર્થીઓ (Negative Numbers) ને પાછળ મોકલવાના છે.

આપણે આ કામ **Two Pointers** (બે પોઈન્ટર) પદ્ધતિનો ઉપયોગ કરીને **O(N)** સમય (Time Complexity) માં કરી શકીએ છીએ.

લોજિક (Logic in Gujarati)

1. આપણે એક પોઈન્ટર રાખીશું જેને નવું નામ આપીશું `posIndex`. આ પોઈન્ટર એ જગ્યા બતાવશે જ્યાં સુધી પહોંચી પાસ થયેલા વિદ્યાર્થી એરેનો પહેલો. શરૂઆતમાં તે `0` પર હશે.
2. ત્યારબાદ આપણે આખા એરે પર લૂપ ફેરવીશું (એરેનો પોઈન્ટર `i`).
3. જો આપણને કોઈ પાસ થયેલો વિદ્યાર્થી (>= 0) મળે:
   - તો આપણે તેને `posIndex` વાળી જગ્યા પર લઈ જઈશું (Swap કરીશું).
   - અને `posIndex` ને એક ડગલું આગળ વધારીશું.
4. જો નાપાસ વિદ્યાર્થી (< 0) મળે, તો આપણે કંઈ કરવાનું નથી, બસ આગળ વધવાનું છે.

નોંધ: આ રીતની પાસ થયેલા વિદ્યાર્થીઓનો ક્રમ જળવાઈ રહેશે, પાસ નાપાસ વિદ્યાર્થીઓનો ક્રમ બરાબર્ સ્ટર્ડ છે. પહેલે, બધા નાપાસ વિદ્યાર્થીઓ છેલ્લે જતા રહેશે.

JavaScript Code

```javascript
function reorderStudents(arr) {
    let n = arr.length;
    let posIndex = 0; // આ પોઈન્ટર પાસ થયેલાના ક્રમાંક માટે છે

    // મુખ્ય એરેમાં ફરો
    for (let i = 0; i < n; i++) {

        // જો વિદ્યાર્થી પાસ છે (સ્કોર >= 0)
        if (arr[i] >= 0) {

            // તો તેને posIndex વાળી જગ્યા સાથે અદલા-બદલી (Swap) કરો
            let temp = arr[posIndex];
            arr[posIndex] = arr[i];
            arr[i] = temp;

            // હવે પહેલી પાસ વિદ્યાર્થી આગળની સીટ પર બેઠશે
            posIndex++;
        }
    }

    return arr;
}

// ઉદાહરણ (Example)
let scores = [1, -1, 3, 2, -7, -5, 11, 6];
console.log("Input: ", scores);

let result = reorderStudents(scores);
console.log("Reordered:", result);

// Output : [ 1, 3, 2, 11, 6, -1, -5, -7 ]
// (નોંધ: નાપાસ વિદ્યાર્થીઓ ઉમેરો છે, જ્યારે તેમનો ક્રમ બદલાશે)
```

સ્ટેપ-બાય-સ્ટેપ ડ્રાય રન (Dry Run)

Input: `[1, -1, 3, 2, -7]` (સરળતા માટે નાનું ઉદાહરણ)

- **શરૂઆત:** `posIndex = 0`

1. **i = 0 (Value: 1):**
   - અંક Positive છે? હા.
   - Swap `arr[0]` અને `arr[0]` (પોતાની સાથે જ).
   - `posIndex` વધીને 1 થશે.
   - Array: `[1, -1, 3, 2, -7]`
2. **i = 1 (Value: -1):**
   - અંક Positive છે? ના.
   - કંઈ નહીં કરો. `posIndex` હજુ 1 જ છે.
3. **i = 2 (Value: 3):**
   - અંક Positive છે? હા.
   - Swap `arr[2]` (3) અને `arr[1]` (-1).
   - `posIndex` વધીને 2 થશે.
   - Array: `[1, 3, -1, 2, -7]` (જુઓ, 3 આગળ આવી ગયો)
4. **i = 3 (Value: 2):**
   - અંક Positive છે? હા.
   - Swap `arr[3]` (2) અને `arr[2]` (-1).
   - `posIndex` વધીને 3 થશે.
   - Array: `[1, 3, 2, -1, -7]`
5. **i = 4 (Value: -7):**
   - Negative છે. કંઈ નહીં કરો.

Final Result: `[1, 3, 2, -1, -7]` તમે જોઈ શકો છો કે બધા Positive નંબરો આગળ આવી ગયા છે અને Negative નંબરો પાછળ પહેલાથી ગયા છે.

Given a string, create an array of size 26, where each element represents the frequency of a character from 'a' to 'z' in the given string.

Constraints:
1 <= n <= 10^5
The input string consists only of lowercase English letters.

Input Format:
First line containing n length of string
Second line containing a string of lowercase English letters.

Output Format:
An array of size 26, where the i-th element represents the frequency of the (i+1)-th character in the English alphabet.

Examples:
Input:
abracadabra

Output:
[
5, 2, 1, 1, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 2, 0, 0, 0, 0, 0, 0, 0,
0, 0
]

Show thinking

લોજિક, આ પ્રોબ્લેમમાં આપણે ગણતરી કરવાની છે કે સિદ્ધિમાં 'a' થી 'z' સુધીના દરેક અક્ષરો કેટલી વાર આવે છે (Frequency Count).

એરેમાંનું કુલ 26 નાના અક્ષરો (lowercase letters) હોય છે, તેથી આપણે **26 શ્રાઈન્ડ એક એરે (Array)** બનાવીશું જેમાં આપણે ગણતરી કરીશું.

લોજિક (Logic in Gujarati)

1. **શરૂઆત (Setup):** આપણે `26` ખાના વાળો એક એરે બનાવીશું અને શરૂઆતમાં બધામાં `0` ભરી દઈશું. (કેમ કે ગણતરી 0 થી શરૂ થાય).
2. **ઈન્ડેક્સ નક્કી કરવા (Mapping):** કમ્પ્યૂટરમાં દરેક અક્ષરના અંક નંબર (ASCII Code) હોય છે.
   - 'a' નો કોડ **97** છે.
   - 'b' નો કોડ **98** છે.
   - જો આપણે 'a' માટે 0 ઈન્ડેક્સ જોઈતો હોય, તો આપણે **97 - 97 = 0** કરવું પડે.
   - જો આપણે 'b' માટે 1 ઈન્ડેક્સ જોઈતો હોય, તો **98 - 97 = 1** કરવું પડે.
   - **સૂત્ર:** `index = charCode - 97`..
3. **લૂપ અને ગણતરી (Loop & Count):**
   - સિદ્ધિના દરેક અક્ષર પર ફરો.
   - તેનો કોડ શોધો અને તેમાંથી 97 બાદ કરો.
   - જે ઈન્ડેક્સ મળે, તે ખાનામાં `+1` ઉમેરો.

JavaScript Code

```javascript
function charFrequency(str) {
    // 1. 26 ઈન્ડેક્સનો એરે બનાવો અને બધામાં 0 ભરો
    // આ એરે [0, 0, 0, ... 0] જેવો દેખાશે
    let frequency = new Array(26).fill(0);

    // 2. સિદ્ધિ પર લૂપ ફેરવો
    for (let i = 0; i < str.length; i++) {

        // અક્ષરનો ASCII કોડ શોધવો (ઉદાહરણ: 'a' હોય તો 97, 'b' હોય તો 98)
        let asciiCode = str.charCodeAt(i);

        // 3. ઈન્ડેક્સ શોધવા માટે 97 બાદ કરો
        // 'a' -> 97 - 97 = 0
        // 'z' -> 122 - 97 = 25
        let index = asciiCode - 97;

        // 4. તે ઈન્ડેક્સના પર ગણતરી 1 વધારો
        frequency[index]++;
    }

    return frequency;
}

// ઉદાહરણ (Example)
let inputStr = "abracadabra";
let result = charFrequency(inputStr);

console.log(result);
```

ઉદાહરણ સાથે સમજૂતી (Dry Run: "bad")

ધારો કે શબ્દ "bad" છે.

1. **શરૂઆત:** `frequency` એરેમાં બધા 0 છે.
2. **પહેલો અક્ષર 'b' (i=0):**
   - 'b' નો કોડ = 98.
   - ઈન્ડેક્સ = 98 - 97 = 1.
   - એરેના 1 નંબરના ખાનામાં +1 કરો.
   - `frequency` = [0, 1, 0, 0, ... 0] ('b' ગણાયો).
3. **બીજો અક્ષર 'a' (i=1):**
   - 'a' નો કોડ = 97.
   - ઈન્ડેક્સ = 97 - 97 = 0.
   - એરેના 0 નંબરના ખાનામાં +1 કરો.
   - `frequency` = [1, 1, 0, 0, ... 0] ('a' અને 'b' ગણાયા).
4. **ત્રીજો અક્ષર 'd' (i=2):**
   - 'd' નો કોડ = 100.
   - ઈન્ડેક્સ = 100 - 97 = 3.
   - એરેના 3 નંબરના ખાનામાં +1 કરો.
   - `frequency` = [1, 1, 0, 1, ... 0].

Output: `[1, 1, 0, 1, 0, ... 0]` (જે-નો અર્થ: 1 વાર 'a', 1 વાર 'b', 0 વાર 'c', 1 વાર 'd'...)

આ રીત **O(N)** ટાઈમ લેશે, એટલે કે સિદ્ધિ જેટલી લાંબી હોય તેટલો સમય લાગશે, જે ખૂબ ઝડપી ગણાય.

Gemini can make mistakes, so double-check it